Detailed Design Specification (DDS)

Demand Forecasting and Restock Advisor for Small Shops (working title)

Document Information

- Version: 0.1 (draft for team review)
- Date: 26 October 2026
- Based on: SRS v0.1, approved by the team
- Status: Design only. Nothing described here has been implemented or tested. No code will be written until this DDS is approved.
- Notation: IDs from the SRS (FR, NFR, ML, BE, DB, FE, SEC, AC) are used for traceability. [PROPOSED] marks a design choice that settles an SRS open decision (D-xx) and needs the team's confirmation. Appendix A lists all of them.
- Dataset facts used here come from checks run on the team's sales_data.csv: 76,000 rows, 16 columns, 2022-01-01 to 2024-01-30 (760 days), 5 stores x 20 products = 100 series, each with all 760 days, no nulls, no duplicates.

1. System Architecture and Technology Stack

1.1 Architecture style

The system has four parts that all run on one laptop: a browser frontend, a Spring Boot REST API, a MySQL database and a separate Python ML service.

- The browser talks only to the Spring Boot backend.
- The backend validates uploads, stores all business data in MySQL, applies the restock rules and calls the Python ML service over HTTP with JSON for training and prediction.
- The ML service keeps no business data. It keeps only saved model files and a small recent-history snapshot per model.

Why this shape: the course stack requires a Java backend, while scikit-learn and pandas are Python libraries. A small HTTP boundary keeps each side in its own language, lets each be tested alone, and keeps ML code reusable from Colab notebooks.

1.2 Technology stack

- Frontend: HTML5, CSS3, plain JavaScript (ES6, no framework). Chart.js 4.x for charts, saved as a local file in the project so no internet is needed at run time.

- Backend: Java 17, Spring Boot 3.x (Web, Validation, Data JPA), Apache Commons CSV for parsing, JdbcTemplate for bulk inserts, Maven.
- Database: MySQL 8.x with the mysql-connector-j driver.
- ML: Python 3.10 or later, pandas, NumPy, scikit-learn, Matplotlib and Seaborn (analysis only), joblib (saving models).
- ML service: FastAPI with uvicorn and pydantic [PROPOSED, D-01].
- Tools: VS Code, Git and GitHub, Google Colab (experiments and notebooks for the ML phases only), Postman or curl, pytest, JUnit 5.
- Additions beyond the SRS stack list that need approval: FastAPI, uvicorn, pydantic, joblib, Apache Commons CSV, Chart.js, pytest, JUnit 5.

1.3 Key technical decisions and alternatives

- **Java to Python link (D-01) [PROPOSED]:** a FastAPI service called by Spring Boot over HTTP/JSON. Alternatives: (a) Spring launching Python scripts with ProcessBuilder, simple at first but reloads the libraries on every call and is fragile with paths and errors; (b) exporting models to ONNX and running them in Java, which removes Python at run time but means rewriting the feature code; (c) Flask instead of FastAPI, equally valid, but FastAPI validates requests automatically and gives interactive API docs; (d) gRPC or a message queue, unnecessary for a single-machine project.
- **Models (D-02) [PROPOSED]:** two baselines (naive same-weekday-last-week, 7-day moving average) plus three scikit-learn candidates (Ridge, RandomForestRegressor, HistGradientBoostingRegressor). ARIMA/SARIMA is not in the core design: our dataset shows almost no weekday pattern (average units sold by weekday ranged only 87.9 to 89.6), and it would add the statsmodels library and one model fit per series. It can still be compared in a notebook.
- **Training strategy (D-08) [PROPOSED]:** train on demand for each upload with one global model across all series of that upload, started by an explicit Train step. Alternatives: a pretrained model, which cannot know an unseen real shop's products; per-series ARIMA, which means about 100 separate fits and an extra library.
- **Forecast method [PROPOSED]:** direct multi-step. One model learns from rows of (forecast origin day, steps ahead h) with h as an input feature, for h = 1 to 14. Alternatives: recursive forecasting, which feeds predictions back in and accumulates error; one model per h, which means 14 models.
- **Frontend hosting [PROPOSED]:** static files served by Spring Boot itself (one origin, no CORS needed in normal use). During development, a CORS rule allows http://localhost:5500 for the VS Code Live Server.

- **Bulk inserts [PROPOSED]:** JdbcTemplate batch inserts for sales rows. Spring Data JPA saveAll with auto-increment keys inserts row by row and is slow for tens of thousands of rows. JPA is still used for all other tables.
- **Training call style [PROPOSED]:** one synchronous request with a loading indicator. If measured training time exceeds the limit set in NFR-02 (to be added, see Appendix B), switch to a start-and-poll design (risk R-01).

2. Architecture Diagram

Figure 1 shows the running parts, their default ports and the direction of every call. The browser never talks to MySQL or to the ML service directly, and the ML service never calls the backend.

```
flowchart LR
  subgraph Client[Browser]
    UI[HTML CSS JS pages<br/>Upload, Forecast, History, Help]
  end
  subgraph Backend[Spring Boot backend, port 8080]
    STATIC[Static file hosting]
    API[REST controllers]
    SVC[Services<br/>validation, training, forecast, restock rules]
    REPO[Repositories<br/>JPA and JdbcTemplate]
    MLC[ML client]
  end
  DB[(MySQL 8, port 3306)]
  subgraph ML[Python ML service, port 8000]
    FAPI[FastAPI endpoints<br/>health, train, predict]
    PIPE[restock_ml package<br/>clean, features, models, evaluate]
    ART[(Model files, joblib)]
  end
  STATIC --> UI
  UI -->|HTTP JSON and CSV upload| API
  API --> SVC
  SVC --> REPO
  REPO --> DB
  SVC --> MLC
  MLC -->|HTTP JSON| FAPI
  FAPI --> PIPE
  PIPE --> ART
```

Figure 1. System architecture.

3. Component and Module Descriptions

3.1 Frontend (static files served by Spring Boot)

- `upload.html` + `js/upload.js`: file picker, a checkbox saying the file is demo/synthetic data, client-side checks, upload summary and error list, and the Prepare forecasting model button with its metrics table.
- `forecast.html` + `js/forecast.js`: choose upload, store, products and horizon; shows forecast table and chart per product; restock form and suggestion cards.
- `history.html` + `js/history.js`: lists of past uploads and forecast runs, each opening its stored result.
- `help.html`: template download, column rules, limits, and the synthetic-data notice.
- `js/api.js` (fetch wrappers that turn error JSON into one error object), `js/ui.js` (loading state, error banner, number and date formatting), `css/styles.css`, `js/vendor/chart.umd.min.js` (Chart.js saved locally).

3.2 Backend (package com.restockadvisor)

- `controller`: HealthController, TemplateController, UploadController, ForecastController, RestockController. Controllers only map HTTP to service calls.
- `service`: UploadService (parse, validate, store), ModelService (train and read model runs), ForecastService, RestockService (pure calculation), HistoryService.
- `repository`: Spring Data JPA repositories for each table, plus SalesRecordBulkRepository (JdbcTemplate batch insert) and a query class that reads series for the ML client.
- `entity`: JPA entities, one per table.
- `dto`: request and response records (Java 17 records) with Bean Validation annotations.
- `client`: MlClient (Spring RestClient) and the records for the ML service JSON.
- `config`: MlProperties (base URL, timeouts), WebConfig (dev-only CORS), upload size settings.
- `exception`: ApiException (HTTP status + error code) and GlobalExceptionHandler (@RestControllerAdvice) producing the common error body.

3.3 ML component

- Package `restock_ml` (plain Python, usable from Colab notebooks and from the service): `ingest.py` (check and normalise incoming records), `clean.py` (build daily series, duplicates, gaps, stock-out flags), `features.py`, `baselines.py`, `models.py`

(candidates and tuning grids), `evaluate.py` (rolling-origin evaluation and metrics), `train.py` (orchestrates a training run), `predict.py`, `artifacts.py` (save and load).

- `service/main.py` and `service/schemas.py`: FastAPI app with pydantic request and response models. It only translates HTTP to calls into `restock_ml`.

3.4 Database

MySQL schema `restock_advisor`, created from `db/schema.sql`. Spring is set to validate the schema against the entities at startup (ddl-auto=validate) and never to create or change tables itself, so the SQL file stays the single source of truth.

4. Detailed Module Designs

4.1 Frontend design

- **Upload page.** After a file is chosen, check the extension is .csv and the size is at most 20 MB before sending. On success show rows stored and rejected, merged duplicates, stores, products, date range and any warnings. If rows were rejected, show up to the first 20 problems with row number, column and reason. Then show the Prepare forecasting model button. While training runs, disable the button and show a progress message. On success show the metrics table (candidate, validation MAE, test MAE, test RMSE, selected marker) and one sentence saying whether the selected model beat the best baseline on the test period.
- **Forecast page.** Controls: upload, store (shown only if the upload has more than one store), products (multi-select, 1 to 20), horizon (7 or 14, default 7). Results per product: a table of forecast dates and predicted units, and a line chart with the last 28 days of history solid and the forecast dashed. A restock form under each result asks for current stock (pre-filled from the last inventory value when the upload has one), lead time in days (1 up to the horizon, default 3) and service level (90, 95, 97.5 or 99 percent, default 95). The suggestion card shows suggested quantity, forecast demand over the lead time, safety stock, current stock, a plain sentence explaining them, and a stock-out risk label.
- **History page.** Two tables, uploads and forecast runs. Selecting a row opens its stored result on the Forecast page without recomputing.
- **States.** Every request shows a loading state, every failure shows the error message from the API in a banner, and every list has an empty-state message such as No uploads yet.
- **Responsive and accessible.** CSS flex and grid layout with breakpoints near 600 px and 900 px, base font at least 16 px, labels tied to inputs, visible keyboard focus, sufficient colour contrast. Each chart has an aria-label and its data is also in the table beside it.

- **Synthetic notice.** If an upload was marked synthetic, a banner says the data is demo data and results are not real-world accuracy (FE-07, NFR-08).
- **Client validation rules.** Horizon is 7 or 14; lead time is a whole number from 1 to the horizon; current stock is a whole number of 0 or more; service level comes from the list; 1 to 20 products. The server repeats every check.

4.2 Backend design

UploadService.handleUpload(file, isSynthetic)

1. Reject files over 20 MB (HTTP 413) and files that are not CSV (HTTP 415).
2. Read as UTF-8 (strip a BOM), parse the header with Apache Commons CSV and normalise names: trim, lower-case, spaces and hyphens to underscores, then map aliases (Store ID to store_id, Product ID to product_id, Units Sold to units_sold, Inventory Level to inventory_level). The Kaggle file therefore works unchanged. If a required column is missing, reject with VALIDATION_FAILED naming it.
3. Validate each row (rules in Section 12). Count all problems, keep the first 20 for the response, and build the valid rows in memory.
4. If more than 5 percent of rows are invalid, or no valid rows remain, reject the whole file and store nothing [PROPOSED, D-13].
5. Merge duplicate (date, store, product) rows: add units sold, keep the last inventory value, take the maximum promotion flag, and count the merges.
6. In one database transaction, insert the upload row, the stores and products, and the sales rows with JdbcTemplate batchUpdate in batches of 1,000 (JDBC URL option rewriteBatchedStatements=true). Any failure rolls everything back.

ModelService.train(uploadId): read the upload's sales rows ordered by store, product and date; send them to the ML service; save the returned run and metrics in one transaction; return the summary. The latest READY run for an upload is the one used for forecasts.

ForecastService.generate(request): check the upload exists and has a READY model run (else 409 MODEL_NOT_TRAINED); resolve the series (if the upload has several stores and none is given, reject with STORE_REQUIRED); call the ML service; save forecast run, series and values; load the last 28 days of history for each series from sales_records; return everything. Series that cannot be forecast are listed under skipped with a reason. If all are skipped, return 422 INSUFFICIENT_HISTORY.

RestockService.compute(...) is a pure function with no database or network access, so it is easy to unit test:

- forecast demand over the lead time = sum of the first L daily forecasts (L = lead time in days)

- safety stock = z x residualStd x square root of L, with z = 1.2816, 1.6449, 1.9600 or 2.3263 for 90, 95, 97.5 or 99 percent service level
- suggested quantity = max(0, round up of (forecast demand over the lead time + safety stock - current stock))
- stock-out risk = current stock is below forecast demand over the lead time
- L must not exceed the horizon (else 422 LEAD_TIME_EXCEEDS_HORIZON)
- Worked example for a test case (illustrative numbers, not results): L = 3, daily forecasts 90, 85, 88 give 263; residualStd 20 gives safety stock 1.6449 x 20 x 1.7321 = 56.98; current stock 120 gives suggested quantity round up of (263 + 56.98 - 120) = 200.

The safety stock formula assumes the daily forecast errors are independent. That is a simplification and is listed as a limitation.

Cross-cutting. Constructor injection, DTOs at the API boundary (entities are never returned directly), Bean Validation on requests, one GlobalExceptionHandler, SLF4J logging, configuration through application.properties and environment variables.

4.3 ML module design

Summarised here and specified in Sections 11 to 13. The service exposes three operations: health, train (takes an upload's cleaned rows, returns a model reference with metrics) and predict (takes a model reference, a horizon and a list of series, returns forecasts). All logic lives in `restock_ml`, so the same code runs in Colab during experiments and behind the service in the final system.

4.4 Database module design

Summarised here and specified in Sections 8 and 9: 11 tables, with the sales table keyed by upload, store, product and date.

5. UML Diagrams

5.1 Sequence diagram: upload

```
sequenceDiagram
  actor Owner as Shop owner
  participant B as Browser
  participant C as UploadController
  participant S as UploadService
  participant D as MySQL
  Owner->>B: Choose CSV and submit
  B->>C: POST /api/uploads, multipart
  C->>S: handleUpload(file, isSynthetic)
  S->>S: Check size, type, header and rows
  alt file rejected
    S-->>C: ApiException VALIDATION_FAILED
    C-->>B: 400 error body with details
  else file accepted
    S->>D: One transaction: upload, stores, products, sales rows
    D-->>S: Committed
    S-->>C: Upload summary
    C-->>B: 201 summary
  end
```

Figure 2. Upload sequence (FR-01 to FR-06).

5.2 Sequence diagram: prepare the forecasting model

```
sequenceDiagram
  participant B as Browser
  participant M as ModelService
  participant D as MySQL
  participant L as MlClient
  participant P as ML service
  B->>M: POST train request for one upload
  M->>D: Read sales rows of the upload
  D-->>M: Rows
  M->>L: train(rows)
  L->>P: POST /v1/models/train
  P->>P: Clean, build features, split, fit, evaluate, select, refit, save
  P-->>L: modelRef, metrics, selected model
  L-->>M: Train result
  M->>D: Save model run and metrics
  M-->>B: 200 model run summary
```

Figure 3. Training sequence (ML-07, ML-09, ML-10).

5.3 Sequence diagram: forecast and restock suggestion

```
sequenceDiagram
  participant B as Browser
  participant F as ForecastService
  participant D as MySQL
  participant L as MlClient
  participant P as ML service
  participant R as RestockService
  B->>F: POST /api/forecasts
  F->>D: Find latest READY model run
  alt no model run
    F-->>B: 409 MODEL_NOT_TRAINED
  else model ready
    F->>L: predict(modelRef, series, horizon)
    L->>P: POST /v1/models/MODEL_REF/predict
    alt ML service down or timeout
      L-->>F: Error
      F-->>B: 503 or 504 error body
    else ML service answered
      P-->>L: Forecasts and residualStd per series
      F->>D: Save forecast run, series and values
      F->>D: Read last 28 days of history
      F-->>B: 201 forecast with history
    end
  end
  B->>R: POST /api/restock-suggestions
  R->>D: Read forecast values and residualStd
  R->>R: Demand over lead time, safety stock, quantity
  R->>D: Save suggestions
  R-->>B: 201 suggestions with explanation
```

Figure 4. Forecast and restock sequence (FR-09 to FR-17, FR-19).

5.4 Class diagram: backend core

```
classDiagram
  class UploadController
  class ForecastController
  class RestockController
  class UploadService {
    +handleUpload(file, isSynthetic) UploadSummary
  }
  class ModelService {
```

```
    +train(uploadId) ModelRunSummary
  }
  class ForecastService {
    +generate(request) ForecastResponse
  }
  class RestockService {
    +compute(forecast, stock, leadTime, serviceLevel) Suggestion
  }
  class MlClient {
    +train(request) MlTrainResponse
    +predict(modelRef, request) MlPredictResponse
  }
  class SalesRecordBulkRepository {
    +insertBatch(rows)
  }
  class ApiException {
    +status
    +code
  }
  class GlobalExceptionHandler
  UploadController --> UploadService
  UploadController --> ModelService
  ForecastController --> ForecastService
  RestockController --> RestockService
  UploadService --> SalesRecordBulkRepository
  ModelService --> MlClient
  ForecastService --> MlClient
  GlobalExceptionHandler ..> ApiException
```

Figure 5. Main backend classes. Repositories for the other tables are Spring Data JPA interfaces and are omitted for readability.

6. Use-Case Diagram

```
flowchart LR
  Owner([Shop owner])
  Eval([Evaluator])
  subgraph System[Restock Advisor]
    UC1((Upload sales CSV))
    UC2((Download CSV template))
    UC3((Prepare forecasting model))
    UC4((Generate forecast))
    UC5((Get restock suggestion))
    UC6((View history))
    UC7((Review model metrics))
```

```
    UC8((Delete an upload, optional))
    UC9((Export suggestions, optional))
  end
  Owner --- UC1
  Owner --- UC2
  Owner --- UC3
  Owner --- UC4
  Owner --- UC5
  Owner --- UC6
  Owner --- UC8
  Owner --- UC9
  Eval --- UC6
  Eval --- UC7
  UC3 -.->|requires| UC1
  UC4 -.->|requires| UC3
  UC5 -.->|requires| UC4
```

Figure 6. Use cases. Dotted arrows show which use case must have happened first.

Mapping to the SRS: UC-01 is Upload sales CSV, UC-02 is Prepare forecasting model plus Generate forecast, UC-03 is Get restock suggestion, UC-04 is View history. The explicit model-preparation step is a design addition (see Appendix B, CR-04).

7. Data-Flow Diagram

Level 0 (context). One process, Restock Advisor. The Shop owner sends a sales CSV, model-preparation requests, forecast requests and restock inputs, and receives upload summaries, metrics, forecasts and suggestions. No other external system is involved, because the system makes no third-party calls (SEC-05).

Level 1.

```
flowchart LR
  OWN[Shop owner]
  P1((1 Validate and store upload))
  P2((2 Prepare model))
  P3((3 Generate forecast))
  P4((4 Compute restock suggestion))
  P5((5 Show history))
  D1[(D1 Sales data, MySQL)]
  D2[(D2 Model runs and metrics, MySQL)]
  D3[(D3 Forecasts and suggestions, MySQL)]
  D4[(D4 Model files, ML service)]
  OWN -->|CSV file| P1
  P1 -->|upload summary and errors| OWN
  P1 -->|validated rows| D1
```

```
  OWN -->|train request| P2
  D1 -->|series rows| P2
  P2 -->|model and recent history| D4
  P2 -->|metrics and model reference| D2
  P2 -->|metrics summary| OWN
  OWN -->|products and horizon| P3
  D1 -->|recent history| P3
  D2 -->|model reference| P3
  D4 -->|model| P3
  P3 -->|forecast values| D3
  P3 -->|forecast and chart data| OWN
  OWN -->|stock, lead time, service level| P4
  D3 -->|forecast and error spread| P4
  P4 -->|suggestions| D3
  P4 -->|quantity and explanation| OWN
  OWN -->|history request| P5
  D1 -->|upload list| P5
  D3 -->|stored results| P5
  P5 -->|history views| OWN
```

Figure 7. Level-1 data-flow diagram.

8. Database Design and ER Diagram

Design rules:

- Everything belongs to one upload, so uploads are isolated from each other and one delete removes all related rows.
- Stores and products are normalised per upload (a code string in the CSV becomes one row).
- Only what the user uploaded is stored in sales_records. Gap-filled days and cleaned series are produced inside the ML service, and the history shown in the UI is read back from sales_records, so nothing is stored twice.
- Only the columns the system uses are stored. Other CSV columns (for example Region, Weather Condition, Units Ordered, Competitor Pricing, Seasonality, Epidemic, Demand) are ignored at upload time (SEC-04).
- Engine InnoDB, character set utf8mb4. CHECK constraints need MySQL 8.0.16 or later.

```
erDiagram
  UPLOADS ||--o{ STORES : has
  UPLOADS ||--o{ PRODUCTS : has
  UPLOADS ||--o{ SALES_RECORDS : contains
  STORES ||--o{ SALES_RECORDS : sells
  PRODUCTS ||--o{ SALES_RECORDS : sold_as
```

```
UPLOADS ||--o{ MODEL_RUNS : trains
MODEL_RUNS ||--o{ MODEL_METRICS : reports
UPLOADS ||--o{ FORECAST_RUNS : forecasts
MODEL_RUNS ||--o{ FORECAST_RUNS : used_by
FORECAST_RUNS ||--o{ FORECAST_SERIES : covers
STORES ||--o{ FORECAST_SERIES : for_store
PRODUCTS ||--o{ FORECAST_SERIES : for_product
FORECAST_SERIES ||--o{ FORECAST_VALUES : has
FORECAST_SERIES ||--o{ RESTOCK_SUGGESTIONS : gets
UPLOADS {
  bigint id PK
  varchar filename
  datetime uploaded_at
  boolean is_synthetic
  date date_min
  date date_max
}
STORES {
  bigint id PK
  bigint upload_id FK
  varchar store_code
}
PRODUCTS {
  bigint id PK
  bigint upload_id FK
  varchar product_code
}
SALES_RECORDS {
  bigint id PK
  bigint upload_id FK
  bigint store_id FK
  bigint product_id FK
  date sale_date
  int units_sold
  int inventory_level
}
MODEL_RUNS {
  bigint id PK
  bigint upload_id FK
  varchar model_ref
  varchar selected_model
  datetime trained_at
}
MODEL_METRICS {
  bigint id PK
  bigint model_run_id FK
```

```
    varchar candidate_name
    decimal test_mae
    decimal test_rmse
  }
  FORECAST_RUNS {
    bigint id PK
    bigint upload_id FK
    bigint model_run_id FK
    tinyint horizon_days
  }
  FORECAST_SERIES {
    bigint id PK
    bigint forecast_run_id FK
    bigint store_id FK
    bigint product_id FK
    decimal residual_std
  }
  FORECAST_VALUES {
    bigint id PK
    bigint forecast_series_id FK
    date forecast_date
    decimal predicted_units
  }
  RESTOCK_SUGGESTIONS {
    bigint id PK
    bigint forecast_series_id FK
    int current_stock
    int suggested_qty
  }
```

Figure 8. ER diagram (key columns only; full column lists are in Section 9).

9. Tables, Columns, Keys and Constraints

All primary keys are `id BIGINT NOT NULL AUTO_INCREMENT`. ON DELETE CASCADE is set on every foreign key, so deleting an upload removes all of its data.

uploads

- filename VARCHAR(255) NOT NULL; uploaded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP; is_synthetic TINYINT(1) NOT NULL DEFAULT 0
- rows_read INT NOT NULL; rows_stored INT NOT NULL; rows_rejected INT NOT NULL; duplicates_merged INT NOT NULL DEFAULT 0
- date_min DATE NOT NULL; date_max DATE NOT NULL; has_inventory TINYINT(1) NOT NULL; has_promotion TINYINT(1) NOT NULL; warnings TEXT NULL

- CHECK rows_stored > 0 and date_min <= date_max

stores

- upload_id BIGINT NOT NULL FK to uploads(id); store_code VARCHAR(50) NOT NULL (value DEFAULT when the file has no store column)
- UNIQUE uq_store (upload_id, store_code)

products

- upload_id BIGINT NOT NULL FK to uploads(id); product_code VARCHAR(50) NOT NULL
- UNIQUE uq_product (upload_id, product_code)

sales_records

- upload_id, store_id, product_id BIGINT NOT NULL, FKs to uploads, stores, products; sale_date DATE NOT NULL
- units_sold INT NOT NULL CHECK (>= 0); inventory_level INT NULL CHECK (>= 0); promotion TINYINT(1) NULL CHECK (0 or 1)
- category VARCHAR(50) NULL; price DECIMAL(10,2) NULL CHECK (>= 0); discount DECIMAL(5,2) NULL CHECK (0 to 100)
- UNIQUE uq_sales (upload_id, store_id, product_id, sale_date); INDEX idx_sales_upload_product_date (upload_id, product_id, sale_date)
- upload_id is repeated on purpose so queries by upload need no join. The service guarantees that the store and product belong to the same upload.

model_runs

- upload_id BIGINT NOT NULL FK to uploads(id); model_ref VARCHAR(64) NOT NULL UNIQUE; trained_at DATETIME NOT NULL
- selected_model VARCHAR(60) NOT NULL; feature_set VARCHAR(10) NOT NULL
- train_start, train_end, val_start, val_end, test_start, test_end DATE NOT NULL
- series_used INT NOT NULL; series_skipped INT NOT NULL; improvement_vs_baseline_pct DECIMAL(7,2) NULL
- INDEX idx_runs_upload (upload_id, trained_at)

model_metrics

- model_run_id BIGINT NOT NULL FK to model_runs(id); candidate_name VARCHAR(60) NOT NULL; kind ENUM('BASELINE','MODEL') NOT NULL; best_params VARCHAR(255) NULL
- val_mae, val_rmse, test_mae, test_rmse DECIMAL(10,4) NOT NULL; is_selected TINYINT(1) NOT NULL

- UNIQUE uq_metric (model_run_id, candidate_name)

forecast_runs

- upload_id BIGINT NOT NULL FK to uploads(id); model_run_id BIGINT NOT NULL FK to model_runs(id)
- horizon_days TINYINT NOT NULL CHECK (7 or 14); created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
- INDEX idx_fr_upload (upload_id, created_at)

forecast_series

- forecast_run_id BIGINT NOT NULL FK to forecast_runs(id); store_id BIGINT NOT NULL FK to stores(id); product_id BIGINT NOT NULL FK to products(id)
- last_observed_date DATE NOT NULL; residual_std DECIMAL(10,4) NOT NULL CHECK (>= 0)
- UNIQUE uq_fs (forecast_run_id, store_id, product_id)

forecast_values

- forecast_series_id BIGINT NOT NULL FK to forecast_series(id); forecast_date DATE NOT NULL; predicted_units DECIMAL(10,2) NOT NULL CHECK (>= 0)
- UNIQUE uq_fv (forecast_series_id, forecast_date)

restock_suggestions

- forecast_series_id BIGINT NOT NULL FK to forecast_series(id); current_stock INT NOT NULL CHECK (>= 0)
- lead_time_days TINYINT NOT NULL CHECK (1 to 14); service_level DECIMAL(4,3) NOT NULL
- forecast_lead_demand DECIMAL(10,2) NOT NULL; safety_stock DECIMAL(10,2) NOT NULL; suggested_qty INT NOT NULL CHECK (>= 0); stockout_risk TINYINT(1) NOT NULL
- created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP. One series may have several suggestions (for example different lead times), so there is no unique key here.

Test data. `db/sample_data.sql` holds a very small upload (2 stores, 2 products, 30 days) for integration tests, produced by a script rather than typed by hand.

10. REST API Specifications

10.1 Conventions

- Base path `/api`. JSON in UTF-8, except the upload (multipart form) and the template and export (CSV). Dates are `yyyy-MM-dd`, timestamps are ISO-8601 in UTC.
- Success codes: 200 OK, 201 Created, 204 No Content. Lists return the newest 100 items first (no paging in the MVP).
- Every error uses one body. Example (format only):

```
{
  "timestamp": "2026-10-26T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "The file has problems that must be fixed",
  "details": [
    { "row": 14, "column": "units_sold", "problem": "must be a whole number,
0 or more" }
  ]
}
```

- Error codes and HTTP status: VALIDATION_FAILED, INVALID_REQUEST, STORE_REQUIRED, CURRENT_STOCK_REQUIRED (400); UPLOAD_NOT_FOUND, PRODUCT_NOT_FOUND, FORECAST_NOT_FOUND, MODEL_NOT_FOUND (404); MODEL_NOT_TRAINED (409); FILE_TOO_LARGE (413); UNSUPPORTED_FILE_TYPE (415); INSUFFICIENT_HISTORY, LEAD_TIME_EXCEEDS_HORIZON (422); INTERNAL_ERROR (500); ML_BAD_RESPONSE (502); ML_SERVICE_UNAVAILABLE (503); ML_SERVICE_TIMEOUT (504).
- Stack traces are never returned (SEC-06). Details are logged on the server.

10.2 Backend endpoints

GET /api/health (BE-07). 200 with backend, database and mlService each UP or DOWN. Used by the Help page and by the test scripts.

GET /api/template (FR-08). 200 CSV file `restock_template.csv`: a header row with date, store_id, product_id, units_sold, inventory_level, promotion, category, price, discount and two example rows.

POST /api/uploads (FR-01 to FR-07). Multipart form with `file` (required) and `isSynthetic` (optional, true or false).

- 201 body fields: id, filename, uploadedAt, isSynthetic, rowsRead, rowsStored, rowsRejected, duplicatesMerged, storeCount, productCount, dateMin, dateMax, hasInventory, hasPromotion, warnings (list of text), errors (up to 20 items of row, column, problem).
- Errors: 400 VALIDATION_FAILED (missing required column, empty file, or more than 5 percent invalid rows), 413 FILE_TOO_LARGE, 415 UNSUPPORTED_FILE_TYPE.

GET /api/uploads (FR-18). 200 list of: id, filename, uploadedAt, isSynthetic, rowsStored, dateMin, dateMax, modelStatus (NOT_TRAINED or READY).

GET /api/uploads/{id} (FR-09, FR-18). 200 with the upload fields above plus storeCodes, productCodes and the latest model run summary (or null). 404 UPLOAD_NOT_FOUND.

DELETE /api/uploads/{id} [O] (FR-20). 204. 404 UPLOAD_NOT_FOUND.

POST /api/uploads/{id}/train (ML-07, ML-10, FR-13). No body. 200 body fields: modelRunId, modelRef, trainedAt, selectedModel, featureSet, splits (trainStart, trainEnd, valStart, valEnd, testStart, testEnd), candidates (list of name, kind, bestParams, valMae, valRmse, testMae, testRmse, isSelected), improvementVsBestBaselinePct, seriesUsed, seriesSkipped (list of storeCode, productCode, reason). Errors: 404 UPLOAD_NOT_FOUND, 422 INSUFFICIENT_HISTORY (no series long enough), 502, 503, 504.

GET /api/uploads/{id}/model (FR-13). 200 with the latest model run summary in the same shape. 404 MODEL_NOT_FOUND.

POST /api/forecasts (FR-09 to FR-14). Request example (format only):

```
{
  "uploadId": 1,
  "storeCode": "S001",
  "productCodes": ["P0001", "P0002"],
  "horizonDays": 7
}
```

Rules: horizonDays is 7 or 14; 1 to 20 productCodes; storeCode may be omitted only when the upload has one store. Response 201 (format only, numbers are placeholders):

```
{
  "forecastRunId": 0,
  "uploadId": 1,
  "modelRunId": 0,
  "modelName": "<selected model name>",
  "horizonDays": 7,
  "createdAt": "<timestamp>",
  "metrics": { "selectedTestMae": 0.0, "selectedTestRmse": 0.0,
```

```
"bestBaselineTestMae": 0.0 },
  "series": [
    {
      "storeCode": "S001",
      "productCode": "P0001",
      "lastObservedDate": "<date>",
      "residualStd": 0.0,
      "history": [ { "date": "<date>", "unitsSold": 0 } ],
      "forecast": [ { "date": "<date>", "predictedUnits": 0.0 } ]
    }
  ],
  "skipped": [ { "storeCode": "S001", "productCode": "P0002", "reason":
"INSUFFICIENT_HISTORY" } ]
}
```

History holds the last 28 days. Errors: 400 INVALID_REQUEST or STORE_REQUIRED, 404 UPLOAD_NOT_FOUND or PRODUCT_NOT_FOUND, 409 MODEL_NOT_TRAINED, 422 INSUFFICIENT_HISTORY (every requested series skipped), 502, 503, 504.

GET /api/forecasts?uploadId= (FR-18). 200 list of: forecastRunId, uploadId, createdAt, horizonDays, modelName, seriesCount.

GET /api/forecasts/{id} (FR-18). 200 in the same shape as the POST response, rebuilt from the database, plus any saved suggestions. 404 FORECAST_NOT_FOUND.

POST /api/restock-suggestions (FR-15 to FR-17). Request example (format only):

```
{
  "forecastRunId": 0,
  "items": [
    { "storeCode": "S001", "productCode": "P0001", "currentStock": 120,
"leadTimeDays": 3, "serviceLevel": 0.95 }
  ]
}
```

Rules: leadTimeDays from 1 to the run's horizon; serviceLevel is 0.90, 0.95, 0.975 or 0.99 (default 0.95); currentStock is optional only if the upload has an inventory column, in which case the last stored value is used. Response 201: a list of suggestions with id, storeCode, productCode, currentStock, leadTimeDays, serviceLevel, forecastLeadDemand, safetyStock, suggestedQty, stockoutRisk and explanation (one plain sentence). Errors: 400 INVALID_REQUEST or CURRENT_STOCK_REQUIRED, 404 FORECAST_NOT_FOUND, 422 LEAD_TIME_EXCEEDS_HORIZON.

GET /api/restock-suggestions?forecastRunId= (FR-18). 200 list in the same shape.

GET /api/restock-suggestions/export?forecastRunId= [O] (FR-21). 200 CSV. Text cells that start with =, +, - or @ are prefixed with a quote character to prevent spreadsheet formula injection.